Automata Theory

Context Free Languages

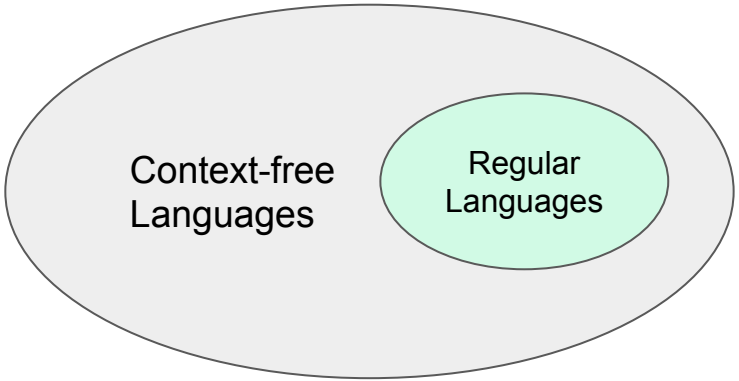
CS32022

By Thisara Watawana

23 September 2026

6. Context Free Languages

- A Context-Free Language (CFL) is a language generated by a Context-Free Grammar (CFG) or, equivalently, recognized by a Pushdown Automaton (PDA).
- Where finite automata and regular expressions focus on scanning inputs sequentially, grammars shift the paradigm to generating strings through structural substitution rules.
- Context-Free Languages include all the regular languages and many additional languages.



6. Context Free Languages

Context-Free Grammar

A Context-Free Grammar is a set of recursive production rules used to construct valid strings in a language. Formally, a CFG is defined as a 4-tuple:

$$G = (V, \Sigma, R, S)$$

- ***V* (Variables / Non-terminals):** A finite set of syntactic placeholders (e.g., S, A, B or ⟨EXPR⟩) that are replaced during string generation.
- **Σ (Terminals):** A finite set of actual output symbols that make up the final strings (e.g., {0, 1} or {a, b, +, ×}). $V \cap \Sigma = \varnothing$.
- ***R* (Production Rules):** A finite set of mapping rules of the form:
  $A \to w$ where $A \in V$ and $w \in (V \cup \Sigma)^*$
- ***S* ∈ *V* (Start Variable):** The initial variable from which derivations begin.

6. Context Free Languages

Context-Free Grammar

Example: Generating 0^n1^n

- **Variables:** $V = \{S\}$
- **Terminals:** $\Sigma = \{0, 1\}$
- **Rules (R):** $S \to 0S1 \mid \varepsilon$

Derivation for 0011:

$$S \Rightarrow 0S1 \Rightarrow 0(0S1)1 = 00S11 \Rightarrow 00(\varepsilon)11 = 0011$$

Why is it Called "Context-Free"?

The name **Context-Free** comes from the left-hand side constraint of the production rules:

$$\text{Single Variable } A \to \text{Anything } w$$

In a context-free grammar, whenever you see the variable A, you can replace it with w regardless of what symbols appear around A. The surrounding "context" plays zero role in whether the rule applies.

6. Context Free Languages

Property	Regular Languages	Context-Free Languages
Recognizing Machine	Finite Automaton (DFA / NFA)	Pushdown Automaton (PDA)
Generative Grammar	Regular / Right-Linear Grammar	Context-Free Grammar (CFG)
Memory Capability	Finite (State memory only)	Unbounded LIFO (Stack memory)
Example	(01)*	0^n1^n
Nondeterminism Power	{NFA} ={DFA} (Equal power)	{DPDA} ≠ {NPDA} (Strictly unequal)
Practical Application	Lexical Analysis (Tokens/Regex)	Syntax Analysis (Parsers/ASTs)

6. Context Free Languages

Exercise 01

Consider the context-free grammar with rules

$$S \to (S)$$

$$S \to S + S$$

$$S \to a$$

- Construct the derivation and parse tree for the valid input string "(a + a)".
- Substitute the start variable iteratively from left to right until all non-terminals are fully resolved.
- Build a hierarchical tree where internal nodes represent non-terminal variables and leaf nodes contain terminal symbols.

6. Context Free Languages

Exercise 01: Solution

1. Leftmost Derivation for *w* = (a + a)

To derive the string, we replace the leftmost variable at every step:

$S \Rightarrow (S)$ (Apply Rule 1)

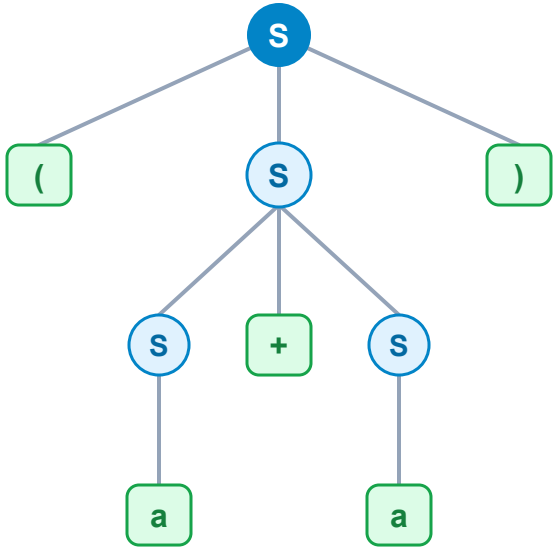
$\Rightarrow (S + S)$ (Apply Rule 2 to S)

$\Rightarrow (a + S)$ (Apply Rule 3 to 1st S)

$\Rightarrow (a + a)$ (Apply Rule 3 to 2nd S)

2. Parse Tree Construction

The parse tree shows the visual structure created by the grammar used to construct Abstract Syntax Trees (ASTs). ⇒



Leaves (L to R): **(, a , + , a ,)** → yields **(a + a)**

6. Context Free Languages

Exercise 01: Solution

Step	Stack Top	Input Symbol	Parser Action	Applied Rule / Result
1	*S*	(	**Expand** variable *S*	*S* → (*S*) (Push), *S*, ()
2	(	(	**Match & Pop** terminal	Input (matched and consumed
3	*S*	a	**Expand** variable *S*	*S* → *S* + *S* (Push *S*, +, *S*)
4	*S*	a	**Expand** variable *S*	*S* → *a* (Push a)
5	a	a	**Match & Pop** terminal	Input a matched and consumed
6	+	+	**Match & Pop** terminal	Input + matched and consumed
7	*S*	a	**Expand** variable *S*	*S* → *a* (Push a)
8	a	a	**Match & Pop** terminal	Input a matched and consumed
9	)	)	**Match & Pop** terminal	Input) matched and consumed
10	$	$	**Accept**	Stack and input empty!

6. Context Free Languages

Exercise 02

Consider the context-free grammar with rules

$$S \rightarrow T + S$$
$$S \rightarrow T$$
$$T \rightarrow a$$

Perform the derivation and trace the top-down parser for the valid input string "a + a + a".

6. Context Free Languages

Exercise 02: Solution

1. Formal Grammar Definition

$G = (\{S, T\}, \{a, +\}, R, S)$

R: $S \to T + S \mid T \qquad T \to a$

2. Leftmost Derivation for w = a + a + a

$S \Rightarrow T + S$ (Apply Rule 1)

$\Rightarrow a + S$ (Apply Rule 3 to T)

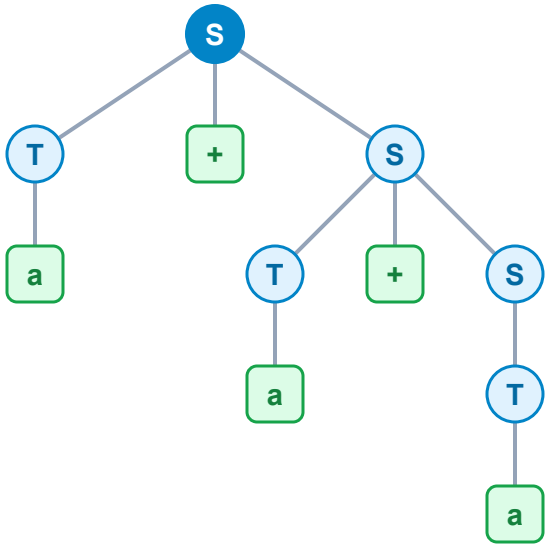
$\Rightarrow a + T + S$ (Apply Rule 1 to S)

$\Rightarrow a + a + S$ (Apply Rule 3 to T)

$\Rightarrow a + a + T$ (Apply Rule 2 to S)

$\Rightarrow a + a + a$ (Apply Rule 3 to T)

3. Parse Tree Construction



Yield (Leaves L to R): **a , + , a , + , a** yields **a + a + a**

6. Context Free Languages

Step	Stack Top	Input Symbol	Parser Action	Applied Rule / Result
1	*S*	a	**Expand** variable *S*	*S* → *T* + *S* (Push *S*, +, *T*)
2	*T*	a	**Expand** variable *T*	*T* → a (Push a)
3	a	a	**Match & Pop** terminal	Input a matched and consumed
4	+	+	**Match & Pop** terminal	Input + matched and consumed
5	*S*	a	**Expand** variable *S*	*S* → *T* + *S* (Lookahead sees more input)
6	*T*	a	**Expand** variable *T*	*T* → a (Push a)
7	a	a	**Match & Pop** terminal	Input a matched and consumed
8	+	+	**Match & Pop** terminal	Input + matched and consumed
9	*S*	a	**Expand** variable *S*	*S* → *T* (Final term in stream)
10	*T*	a	**Expand** variable *T*	*T* → a (Push a)
11	a	a	**Match & Pop** terminal	Input a matched and consumed
12	$	$	**Accept**	Stack and input empty!

6. Context Free Languages

Grammar Ambiguity

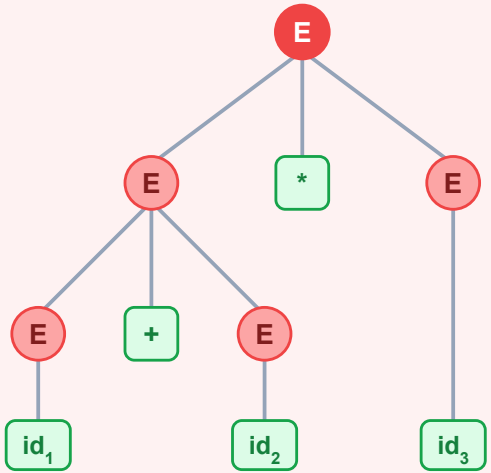
- A context-free grammar is ambiguous if at least one string in its language has two or more distinct leftmost derivations.
- Multiple derivations for a single input string produce distinctly shaped parse trees representing conflicting structural interpretations.
- Compilers assign semantic meaning directly from parse tree shapes, so structural ambiguity creates undefined computation behavior.
- **Arithmetic Precedence Failure**: An unconstrained expression grammar generates competing parse trees that evaluate addition before multiplication or vice versa.
- Ambiguous grammars must be restructured into unambiguous forms before they can be used in reliable compiler parsers.

6. Context Free Languages

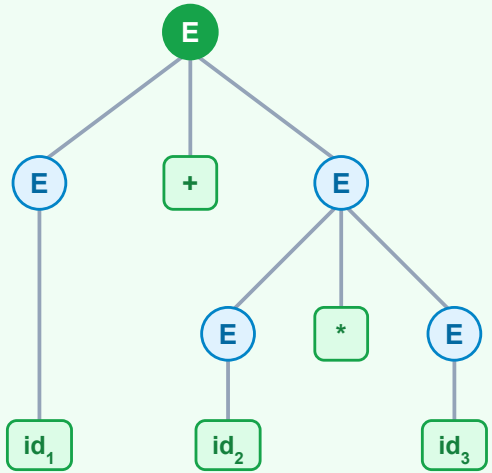
Grammar Ambiguity

1. Ambiguous Grammar (*G1*): $E \rightarrow E + E \mid E \times E \mid id$
2. Target String: $w = id_1 + id_2 \times id_3$
3. Two Competing Structural Parse Trees

Tree A: $(id_1 + id_2) * id_3$ [INCORRECT]



Tree B: $id_1 + (id_2 * id_3)$ [CORRECT]



6. Context Free Languages

Grammar Ambiguity: Disambiguating Expression Grammars

Precedence Level Stratification: Separate non-terminal variables into hierarchical layers to enforce strict operator evaluation order.

- **Top Expression Layer:** Define top-level expression rules to process lowest-precedence operators like addition and subtraction.
- **Intermediate Term Layer:** Define term-level rules to handle higher-precedence operators like multiplication and division.
- **Atomic Factor Layer:** Enforce factor-level rules to handle atomic variables, constant numbers, and parenthesized expressions.

6. Context Free Languages

Grammar Ambiguity: Disambiguating Expression Grammars

- **Associativity Control:** Position non-terminals recursively on the left or right side of rules to mandate evaluation direction.

The Stratified Unambiguous Grammar ($G2$):

$E \rightarrow E + T \mid T$ (Expression layer: handles +, lowest precedence)

$T \rightarrow T \times F \mid F$ (Term layer: handles , higher precedence)

$F \rightarrow (E) \mid \texttt{id}$ (Factor layer: handles atomic values and parens)

6. Context Free Languages

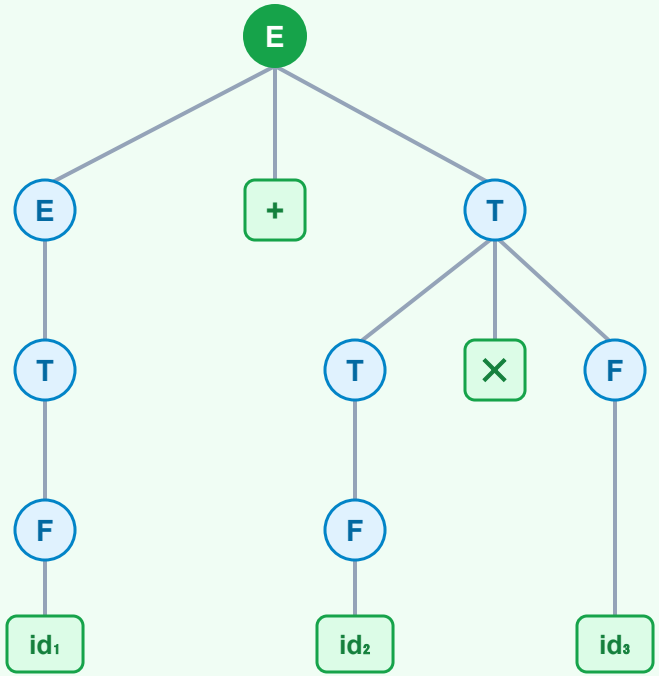
Grammar Ambiguity: Disambiguating Expression Grammars

Re-parsing target string: $w = \mathrm{id}_1 + \mathrm{id}_2 \times \mathrm{id}_3$

Grammar G_2 yields a unique leftmost derivation:

$$
\begin{aligned}
E &\Rightarrow E + T \\
&\Rightarrow T + T \\
&\Rightarrow F + T \\
&\Rightarrow \mathrm{id}_1 + T \\
&\Rightarrow \mathrm{id}_1 + T \times F \\
&\Rightarrow \mathrm{id}_1 + F \times F \\
&\Rightarrow \mathrm{id}_1 + \mathrm{id}_2 \times F \\
&\Rightarrow \mathrm{id}_1 + \mathrm{id}_2 \times \mathrm{id}_3
\end{aligned}
$$

Forced Unique Parse Tree (Grammar G_2) [UNAMBIGUOUS]



6. Context Free Languages

Equivalence of CFGs and PDAs

- A language is context-free if and only if some pushdown automaton recognizes it.
- Every context-free grammar can be converted into an equivalent PDA and vice versa.
- A PDA simulates a CFG by predicting production rules and matching terminal symbols against input.
- The conversion algorithm constructs a 3-state PDA that performs all rule expansions on a single central state.
- This equivalence guarantees that every grammar-based programming language can be parsed using stack-based execution.

6. Context Free Languages

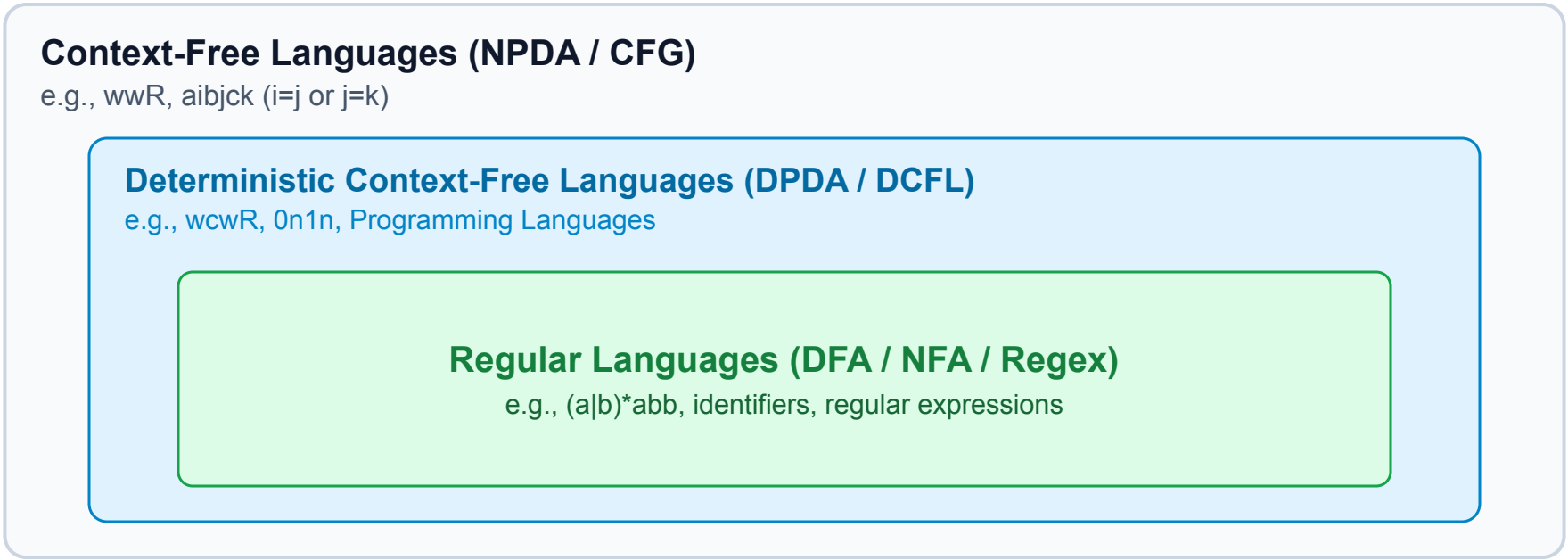
Deterministic PDAs and Compiler Parsing

- Unlike finite automata where DFAs match NFAs, deterministic pushdown automata are strictly less powerful than nondeterministic pushdown automata.
- Deterministic Context-Free Languages form a strict subset of context-free languages that can be recognized without nondeterministic guessing.
- Arbitrary context-free grammars require cubic $O(n^3)$ time algorithms such as CYK or Earley parsing to process input strings.
- Programming language compilers require linear O(n) parsing time to process large source code files efficiently.
- Compiler generators restrict grammars to deterministic subclasses like LL(k) or LR(k) to eliminate backtracking completely.

6. Context Free Languages

The Language Inclusion Hierarchy

`Regular Languages ⊊ Deterministic CFLs (DCFL) ⊊ Context-Free Languages (CFL)`



6. Context Free Languages

Deterministic PDAs and Compiler Parsing

Finite Automata: `DFA = NFA`

- Nondeterminism provides convenience, not additional language recognition power.

Pushdown Automata: `DPDA ⊊ NPDA`

- Nondeterminism provides fundamental structural power.

$L_1 = \{ w c w^r \mid w \in \{a,b\}^* \} \in DCFL$ (Recognized by DPDA because center marker c unambiguously dictates when to transition.)

$L_2 = \{ w w^r \mid w \in \{a,b\}^* \} \in CFL \setminus DCFL$ (Requires an NPDA because guessing the midpoint is fundamentally non-deterministic.)

Questions?

thisaraw@boffin.lk